

CRM Sync Setup Reference

The full technical reference for the CRM Sync stack — every connection (Cloudflare Worker, Xano, Shopify, Google, Resend, Webflow) in depth. **New here? You don't need this page yet** — follow the short numbered [getting-started steps](#) first; come back when you want every detail.

Getting started? Follow the [step-by-step setup at crm-sync.dev/start](#) — a short numbered list that tells you exactly where each key comes from and which screen it goes into. This page is the full reference, for when you want every detail behind those steps.

Everything you need to get your CRM system running. Complete each section in order — each one builds on the last.

What is CRM Sync? It connects six services together: a **server** (Cloudflare Worker) that runs your auth and sync logic, a **database** (Xano) that stores users and tag relationships, a **store** (Shopify) for customer sync, **analytics** (Google GA4) for tracking consent and user segments, **email** (Resend) for transactional emails, and a **CMS** (Webflow) that displays user data and manages campaigns. You configure each one, then the Webflow App ties them together.

Tab 1: Cloudflare Worker

What is this?

The Cloudflare Worker is your **backend server**. It handles:

- **User authentication** — signup, login, password reset, OAuth (Google + Shopify)

- **Consent management** — cookie banners, TOS acceptance, GDPR compliance
- **Customer sync** — bidirectional tag sync between Shopify, Xano, Webflow CMS, and GA4
- **Tag system** — structured CRM tags with categories (status, tier, segment, campaign, consent, marketing)
- **GA4 integration** — pushes user properties and events to Google Analytics via Measurement Protocol
- **Serving UI** — the login modal, consent banner, account page, and UCP dashboard are all served from here

Why Cloudflare? It's fast (runs at the edge, close to your users), has a generous free tier, and includes KV storage for caching config and session data.

► Setup Steps

Alternative: Skip the CLI

If you install the CRM Sync Webflow extension, you can set all credentials from the **Config** tab in the Webflow Designer panel. Values saved there override `wrangler.toml` — so after the initial deploy, you never need the command line again.

Tab 2: Xano (Database)

What is this?

Xano is your **database**. It stores all your user accounts, consent records, profile data, and the CRM tag system. The CRM Worker talks to Xano using the **Meta API**.

What gets stored:

- **User accounts** — email, name, password hash, login provider, Shopify/Google IDs
- **Consent preferences** — which policies each user accepted, when, and how
- **Consent audit log** — a timestamped record of every consent change (required for GDPR)
- **CRM tags** — structured tags with name, slug, and category
- **User-tag assignments** — a join table linking users to tags with source attribution
- **User extras** — a flexible table for any additional data

► Setup Steps

Tab 3: Shopify (Store Integration)

What is this?

The Shopify integration does four things:

1. **Admin API** — Syncs customer data between your CRM and Shopify. Tags, metafields, consent status, and order data are synced in real-time via webhooks and every 15 minutes via cron. Also handles GDPR data requests.
2. **Real-Time Webhooks** — When a customer is created or updated in Shopify, webhooks immediately sync to Xano and Webflow CMS. New Shopify customers automatically receive a welcome email to set their website password.
3. **Customer Account OAuth** — Lets users sign in with their Shopify account using PKCE (no client secret needed).

4. **CRM Tag Sync** — Tags added from the UCP dashboard flow to Shopify as both customer tags (for Shopify Segments/Flows) and structured metafields (for custom reporting).

You need three things from Shopify: a Dev Dashboard app (for Admin API token + app secret), a Headless channel (for OAuth login), and GDPR webhook URLs.

- ▶ **Setup Steps: Dev Dashboard App (Admin API)**
- ▶ **Setup Steps: "Sign in with Shopify" (OAuth)**
- ▶ **Setup Steps: GDPR Compliance Webhooks**

Tab 4: Google (Analytics, OAuth & GA4 Segments)

What is this?

The Google integration has three parts:

1. **Google OAuth** — "Sign in with Google" button via OpenID Connect.
2. **GA4 Consent Mode** — Connects your CRM consent banner to Google Analytics. When a user accepts or rejects cookies, GA4 is updated to respect their choice.
3. **GA4 Measurement Protocol** — Server-side push of CRM user properties to GA4. Every tag change (from the UCP dashboard, Shopify sync, or admin API) pushes structured user properties to GA4, making them available for GA4 audiences, Google Ads audience sharing, and Looker Studio.

GA4 User Properties pushed by the worker:

PROPERTY	SOURCE	EXAMPLE VALUE
<code>crm_status</code>	Status category tags	<code>active</code>
<code>crm_tier</code>	Tier category tags	<code>vip</code>
<code>crm_segment</code>	Segment category tags	<code>high_value, returning</code>
<code>crm_campaign</code>	Campaign category tags	<code>new_campaign, summer_2026</code>
<code>crm_tags</code>	All tag slugs	<code>active, vip, new_campaign</code>
<code>consent_marketing</code>	Consent tags	<code>granted</code> or <code>denied</code>
<code>consent_tos</code>	Consent tags	<code>granted</code> or <code>denied</code>

Events sent:

- `crm_tags_updated` — fired when a user adds/removes tags from the dashboard (includes `tags_added`, `tags_removed`, `campaign_tags` params)
 - `crm_sync` — fired during the 15-minute cron sync from Shopify (includes `source: shopify_cron`)
- ▶ **Setup Steps: Google OAuth ("Sign in with Google")**
 - ▶ **Setup Steps: GA4 Measurement Protocol (Server-Side Segments)**
 - ▶ **Setup Steps: GA4 Consent Mode (Client-Side)**
 - ▶ **Google Tag Manager — not required**
 - ▶ **CRM Form Bridge: E2E Event Tracking (Any Form Type)**

Tab 5: Webflow CMS (Customer Data & Tags)

What is this?

Webflow CMS stores a read-friendly copy of your customer data and CRM tags. This lets you build Webflow pages that display customer profiles, filter by tags, and create dynamic content based on CRM segments.

Two collections are used:

1. **Customers** — synced from Xano on every cron run (name, email, provider, consent status, order data, tags)
2. **CRM Tags** — the tag definitions with categories, referenced by the Customers collection via MultiReference

► Setup Steps

Campaign Tags from Dashboard

When a user adds a tag like "new campaign" from the UCP dashboard, it immediately:

1. Creates the tag in Xano (`crm_tags` table, category: `campaign`)
2. Assigns it to the user in the join table (`user_tag_map`)
3. Pushes it to Shopify as a customer tag + metafield
4. Creates it in the Webflow CRM Tags collection
5. Pushes it to GA4 as a `crm_campaign` user property

No cron wait — the full channel flow happens in one request.

Tab 6: Email (Resend)

What is this?

Resend handles **transactional emails**:

1. **Password reset** — When a user clicks "Forgot password?", the worker sends a reset link (1-hour expiry).
2. **Welcome email** — When a customer is added in Shopify and synced to the CRM, they automatically receive a "Welcome — set up your password" email (24-hour expiry). This lets Shopify-origin users create a password to sign in on the website. The forgot-password flow also detects first-time users and sends the welcome variant instead of the reset variant.

► Setup Steps

Tab 7: Webflow Extension (Config UI)

What is this?

The CRM Sync Webflow extension adds a configuration panel to the Webflow Designer. It lets you manage all credentials, auth settings, consent toggles, and embed codes without touching the CLI.

Config tab — Worker URL, Shopify credentials, Google OAuth, Xano, Resend, Webflow CMS, GA4 **Auth tab** — Toggle auth methods (email/Google/Shopify), session settings, consent & privacy toggles **Embeds tab** — Copy-paste embed codes for footer loader, account page, dashboard, compliance page **Status tab** — Health check, endpoint testing, redirect URIs, privacy API tests, GDPR handler tests

Embed Codes

The extension generates four embed snippets:

EMBED	WHERE TO PASTE	WHAT IT RENDERS
CRM Footer Loader	Site Settings > Footer Code	Login modal, consent banner, session management
Account Page	Account page embed block	User profile view/edit
UCP Dashboard	Dashboard page embed block	Consent status, retarget channels, A/B segment, tags, translation, consent history
Compliance / Privacy	Privacy page embed block	Communication preferences, third-party disclosures, data rights

Tab 8: Agentic Commerce – Entitlements, Consent & Enterprise Add-ons

What is this?

The **value-prop layer**: Agentic Commerce Checkout (who/which agent may transact, and under what consent) plus optional **per-Org enterprise data-layer / ERP add-ons** (Adobe AEP, SAP S/4HANA, NielsenIQ/Circana). It is operated entirely from the `/settings` **console** – no code changes, no deploy. All config saves to KV and takes effect immediately.

The console is double-gated: it sits behind **Cloudflare Access** (your zero-trust login) and the **admin key**. Open it as `https://<worker>/settings?key=<ADMIN_KEY>` after authenticating with Access.

One-time setup

1. **Scoped read key** — mint an `entitlement:read` key in Xano and provide it as `XANO_ENTITLEMENT_KEY`. You can paste it into the console (Xano section → *Entitlement Read Key* → **Set**, masked) or set it as a worker secret. The high-volume entitlement read path uses this scoped key, never the master metadata key.
2. **Signing key** — in the console's **Agentic Commerce Core** card, click **Generate** to create the EdDSA signing key (channels verify entitlement tokens offline with its public half). Use **Rotate** later for a zero-downtime, non-destructive roll.
3. **Channels** — the **Core** card shows each channel's cadence (real-time vs batch), connector status, cursor, and last-synced, with a **Sync** button. Cloudflare/Shopify/GA4 are real-time (project on every change); batch channels reconcile every 15 minutes.

Entitlements & consent

- An entitlement is one record per subject (`tenant` / `user` / `agent`) with `plan_tier`, `features[]`, `caps` (a2a / ap2 / mandate_max_amount / channels), and an explicit `consent` object. It carries a monotonic `state_version`.
- **Consent never regresses:** every channel only moves a subject forward in version, so a slow 15-minute batch system can never overwrite a newer consent the real-time channels already applied. Revoking an entitlement drops every channel's projection and denies checkout offline.
- Consent is projected identically everywhere — granular **GA4 Consent Mode v2**, Shopify metafields, and the add-on connectors.

Enterprise add-ons (per Org)

In the **Enterprise Add-ons** card, configure each integration the Org has entitled (an add-on is active only when its `feature` is in the entitlement's `features[]`):

ADD-ON	FEATURE KEY	POINT THE CONNECTOR AT
Adobe AEP	adobe_aep	your AEP HTTP streaming endpoint
SAP S/4HANA	sap	your OData / integration-middleware endpoint
NielsenIQ / Circana	nielsen	your measurement ingestion endpoint

Each connector is `{ url, key, enabled }`; the worker POSTs the versioned entitlement+consent record there. Onboarding a new Org or stack (Salesforce, Braze, Attentive, ...) is configuration, not code.

Header / Footer embed codes

The console's **Header / Footer Embed Codes** card provides copy-paste snippets: the **Head** snippet sets GA4 Consent Mode defaults; the **Footer** snippet loads the enabled embeds. Paste into Webflow → Site Settings → Custom Code.

Deploy channels & deploy teams

The portal is self-identifying per **deploy channel**, resolved by the hostname you open it on — one worker, three channels:

CHANNEL	HOSTNAME PATTERN	DEFAULT DEPLOY TEAM
Dev	localhost / *dev*	Engineering
Stage	*.workers.dev	Agency Consulting Team
Prod / UAT	production custom domain (crm.story-story.ai)	QA · Product Management / DPO · PMO · Deploy On-Prem

The environment·team banner shows at the top of `/settings` and `/setup`. Rename a channel's team in the **Environment / Deploy Team** card (saved per hostname). These values can later be owned by the Webflow Publish Refactor (the publish

target *is* the environment). The Webflow extension header shows the same chip + a **Config Portal** ↗ link.

Localization (markets & geos)

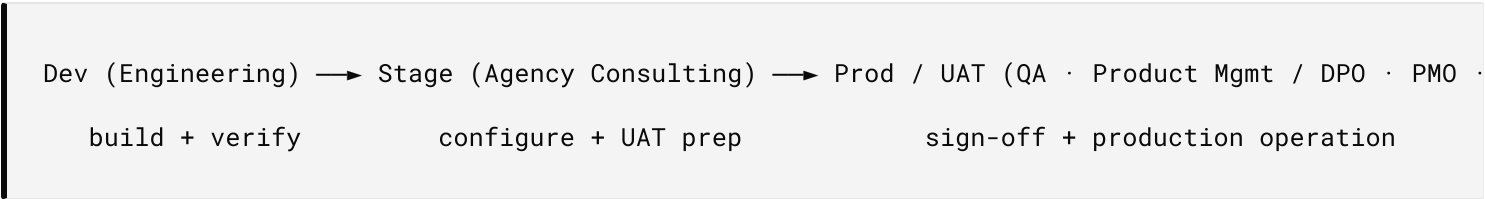
Each tenant has a **market**, grouped by geo for scale:

GEO	MARKETS (LOCALE · CURRENCY)
NA	US en-US·USD · CA en-CA·CAD
EMEA	UK en-GB·GBP · DE de-DE·EUR · FR fr-FR·EUR
APAC	AU en-AU·AUD · NZ en-NZ·NZD · SG en-SG·SGD · JP ja-JP·JPY
LATAM	MX es-MX·MXN · BR pt-BR·BRL

The market is inferred from the shop name (**prefix or suffix:** `us-acme` , `acme-au`) or a country TLD, and overridable in the **Localization (Market)** card (geo-grouped selector). It sets the locale and the **default currency for agent mandates/caps** (e.g. an AU tenant defaults to AUD). Add more markets under any geo without code changes.

Forward-Deploy Harness

Changes move **forward through the three channels**, with each stakeholder operating its own channel from the portal — no code access required downstream:



- Each channel binds its own **Webflow (UI)** and **Xano (Data)** state targets — both shown in the `/settings` env banner (a deploy reaches state in both systems).

- Each channel is operated via `/settings` (KV config, effective on save — no deploy) and gated per stakeholder by **Cloudflare Access** (zero-trust) layered over the admin key.
- Validate any channel with the verification script (use `--read-only` on Stage/Prod to avoid mutations); `/setup` shows Agentic Commerce readiness with a Config Portal deep-link.

Verify

Run `scripts/verify-entitlement.sh` with the admin key to exercise the whole stack end-to-end (entitlement CRUD + state machine, sign/verify/tamper, rotation, channel sync/replay, revoke cascade) with PASS/FAIL output. On Stage/Prod use `--read-only` — it performs no mutations (only reads + a non-mutating verify against the seeded entitlement).

Clean Room note: the privacy-preserving Clean Room (see `CLEAN-ROOM-SPEC.md`) is a *consumer* of this consent — it gates match-job inclusion on `consent.clean_room` at the current version. It is a separate concern (data matching → aggregates), not part of consent propagation.

Pricing Tiers & Upgrade Exceptions

Shared Plan (\$90 one-time download, from \$69) — Stakeholder B

Multi-tenant. You use the hosted CRM Sync worker. API keys are entered via the Webflow extension Config tab and stored in the shared worker's KV (isolated per shop). The Plan tab shows your current plan and active features. Custom Worker Setup is locked.

Billing is via Shopify App Billing — charges appear on your Shopify invoice.

Private Worker Plan (\$325/mo or custom license) – Stakeholder C

You deploy your own Cloudflare Worker instance. **The following UI elements behave differently:**

- **Custom Worker Setup:** Unlocked. You enter your own Worker URL and CDN domain.
- **Plan tab:** Informational only. You already have full access to all features.
- **"Upgrade to unlock" prompts:** Hidden. All integrations (Adobe AEP, custom OAuth domains) are self-managed.
- **Subscription billing:** Not through Shopify App Billing. Billed separately per license agreement.

Fee responsibility on Private Worker plans:

COST	WHO PAYS
CRM Sync license	You (per agreement with App Creator)
Cloudflare Workers + KV	Your Cloudflare account
Xano database	Your Xano account
Shopify API access	Your Shopify Partner/app account
Resend transactional email	Your Resend account
Google OAuth + GA4	Your Google Cloud project
Adobe AEP (if enabled)	Your Adobe contract

To activate Private plan features: Set `plan` to `"private"` in your tenant config:

```
curl -X POST https://YOUR-WORKER.workers.dev/config?shop=YOUR-STORE.myshopify.com \
  -H "Authorization: Bearer YOUR_ADMIN_KEY" \
  -H "Content-Type: application/json" \
  -d '{"plan": "private"}'
```

Self-Service Admin Key Rotation (Persona C)

Private Worker operators can rotate their admin key via API without needing the Cloudflare CLI. This is independent of any Shopify instance – useful when managing multiple Shopify stores from one worker.

Key hierarchy:

- **Root key** (`ADMIN_KEY`): Set once via `wrangler secret put` at deploy time. Cannot be rotated via API.
- **Rotatable key**: Created and managed via `/admin/rotate-key`. Works alongside the root key on all admin endpoints.

Create or rotate a key:

```
# Auto-generate a new key

curl -X POST https://YOUR-WORKER.workers.dev/admin/rotate-key \
  -H "Authorization: Bearer YOUR_CURRENT_KEY" \
  -H "Content-Type: application/json" \
  -d '{}'
```



```
# Or bring your own key (minimum 20 characters)

curl -X POST https://YOUR-WORKER.workers.dev/admin/rotate-key \
  -H "Authorization: Bearer YOUR_CURRENT_KEY" \
  -H "Content-Type: application/json" \
  -d '{"key": "your-custom-key-at-least-20-chars"}'
```

Check current key status:

```
curl https://YOUR-WORKER.workers.dev/admin/rotate-key \  
  -H "Authorization: Bearer YOUR_KEY" \  
# Returns: has_rotatable_key, created_at, rotated_at, rotations, key_preview
```

Revoke rotatable key (root key only):

```
curl -X DELETE https://YOUR-WORKER.workers.dev/admin/rotate-key \  
  -H "Authorization: Bearer YOUR_ROOT_ADMIN_KEY"
```

Important: Rotating the key immediately invalidates the previous rotatable key. The root `ADMIN_KEY` always remains valid. Existing tenant tokens (`crm_t_*`) are not affected by key rotation.

Quick Reference

All credentials in one place. Use the Webflow App Config tab or `wrangler.toml` / `wrangler secret put` .

WHAT	WHERE TO SET	EXAMPLE
Worker URL	Webflow App: Config tab	<code>https://your-worker.workers.dev</code>
Xano API Base URL	Webflow App or <code>wrangler.toml</code>	<code>https://xxx.xano.io/api:XXX</code>
Xano API Key	Webflow App or <code>wrangler secret</code>	<code>eyJhbG...</code>
Google Client ID	Webflow App or <code>wrangler.toml</code>	<code>123456.apps.googleusercontent.com</code>
Google Client Secret	Webflow App or <code>wrangler secret</code>	<code>GOCSPX-xxx</code>
GA4 Measurement ID	Webflow App or <code>wrangler.toml</code>	<code>G-XXXXXXXXXX</code>
GA4 API Secret	Webflow App or <code>wrangler secret</code>	<code>xxxxxxxxx</code>
Shopify Store Domain	Webflow App or <code>wrangler.toml</code>	<code>store.myshopify.com</code>
Shopify Shop ID	Webflow App or <code>wrangler.toml</code>	<code>64312475691</code>
Shopify Client ID	Webflow App or <code>wrangler.toml</code>	<code>890afa5e-...</code>
Shopify Admin Token	Webflow App or OAuth install flow	<code>shpua_xxx</code> (auto via install)
Shopify App Secret	Webflow App or <code>wrangler secret</code>	<code>shpss_xxx</code>
Webflow CMS Token	Webflow App or <code>wrangler secret</code>	<code>xxx...</code>
Webflow Collection ID	Webflow App	<code>xxx...</code>
Resend API Key	Webflow App or <code>wrangler secret</code>	<code>re_xxx</code>
Resend From Email	Webflow App or <code>wrangler.toml</code>	<code>Brand <noreply@domain.com></code>
JWT Secret	<code>wrangler secret</code> only	<code>openssl rand -hex 32</code>

Default CRM Tags

These 17 tags are seeded when you initialize the tag system:

CATEGORY	TAGS
status	Active, Inactive
tier	VIP, Prospect
consent	Accepts Marketing, Rejects Marketing, Accepts TOU, Accepts Privacy, Accepts Cookie
marketing	Email Subscribed, Email Unsubscribed, SMS Subscribed
segment	High Value, Returning, New Customer, At Risk
campaign	Campaign

Tags added from the UCP dashboard that don't match a known category default to **campaign**.
